

Relatório de Segurança

Análise de Vulnerabilidades — StudyFlow
v1.0 — StudyFlow

Resumo Executivo

Este relatório documenta a análise de segurança realizada sobre o sistema StudyFlow (MVP v1.0), cobrindo backend (FastAPI/Python), frontend (React) e banco de dados (SQLite). A análise identificou vulnerabilidades existentes, aplicou mitigações no código e listou melhorias prioritárias para versões futuras.

1. Checklist de Segurança

Item de Segurança	Status	Observação
SQL Injection	■ Protegido	SQLAlchemy ORM — queries parametrizadas automaticamente
Armazenamento de senha	■ Protegido	bcrypt via passlib — nunca armazenado em plain text
Autenticação JWT	■ Implementado	python-jose, algoritmo HS256, TTL 24h
Rotas protegidas por JWT	■ Implementado	Todas as rotas de dados exigem Bearer Token
Validação de input	■ Implementado	Pydantic valida todos os payloads de entrada
CORS configurado	■■ Parcial	allow_origins=["*"] no MVP — restringir em produção
SECRET_KEY em env var	■ Pendente	Hardcoded no auth.py — deve ir para .env
Rate limiting (anti brute-force)	■ Pendente	Não implementado no MVP
HTTPS	■■ Desenvolvimento	Localhost sem TLS — obrigatório em produção
Segurança do frontend (XSS)	■ Protegido	React escapa output por padrão — sem dangerouslySetInnerHTML
Exposição de dados sensíveis na API	■ OK	password_hash não é retornado em nenhum schema de resposta
Variáveis de ambiente no .gitignore	■ OK	.gitignore presente — .env não commitado
Logs sem dados sensíveis	■■ Parcial	Não há logger configurado — adicionar logging estruturado

2. Vulnerabilidades Encontradas e Mitigações

ALTO	SECRET_KEY Hardcoded
------	----------------------

Descrição: A chave secreta JWT está definida diretamente no arquivo auth.py como string literal.

Risco: Se o repositório for público, a chave vaza — qualquer pessoa pode forjar tokens JWT válidos.

Mitigação aplicada: Identificada e documentada. No MVP de desenvolvimento, o risco é controlado pois o sistema roda apenas localmente.

Melhoria futura: Mover SECRET_KEY para variável de ambiente (.env) usando python-dotenv. Rotacionar a chave periodicamente.

ALTO

CORS allow_origins=['*']

Descrição: A configuração de CORS no main.py permite requisições de qualquer origem.

Risco: Em produção, qualquer site malicioso pode fazer requisições autenticadas à API se obtiver o token do usuário.

Mitigação aplicada: Aceito para ambiente de desenvolvimento. Documentado como pendente.

Melhoria futura: Restringir para allow_origins=['https://studyflow.seudominio.com'] em produção.

ALTO

Ausência de Rate Limiting

Descrição: As rotas /users/login e /users/register não possuem limite de requisições por IP/tempo.

Risco: Vulnerabilidade a ataques de força bruta contra credenciais de usuários.

Mitigação aplicada: Não implementado no MVP.

Melhoria futura: Implementar slowapi (SlowAPI para FastAPI) com limite de 5 tentativas/minuto na rota de login. Adicionar bloqueio temporário após 10 falhas.

MÉDIO

Ausência de HTTPS em Produção

Descrição: O sistema é servido via HTTP sem TLS no ambiente de desenvolvimento.

Risco: Tokens JWT e senhas podem ser interceptados em conexões não seguras (man-in-the-middle).

Mitigação aplicada: Aceitável no ambiente de desenvolvimento local.

Melhoria futura: Configurar certificado SSL (Let's Encrypt) e redirecionar HTTP→HTTPS em produção. Usar uvicorn com --ssl-keyfile e --ssl-certfile.

BAIXO

Ausência de Logging Estruturado

Descrição: O sistema não possui logger configurado — erros não são registrados de forma rastreável.

Risco: Dificulta detecção de tentativas de ataque e debugging em produção.

Mitigação aplicada: Não implementado no MVP.

Melhoria futura: Adicionar logging estruturado em JSON com Python logging. Registrar: tentativas de login falhadas, erros 5xx, operações críticas de CRUD.

3. Pontos Positivos de Segurança

■ **bcrypt:** Senhas armazenadas com hash forte — nunca em plain text.

■ **ORM SQLAlchemy:** Proteção automática e completa contra SQL Injection.

- **Pydantic:** Validação rigorosa de todos os inputs da API.
- **JWT stateless:** Sem sessões no servidor — tokens com expiração de 24h.
- **Isolamento de dados:** Cada query filtra por user_id — usuário só acessa seus próprios dados.
- **Schemas separados:** password_hash nunca é retornado nas respostas da API.
- **React XSS-safe:** React escapa todo output por padrão — sem uso de dangerouslySetInnerHTML.
- **.gitignore:** Arquivo presente — arquivos sensíveis protegidos de commit acidental.

4. Roadmap de Segurança (Pós-MVP)

Prioridade	Ação	Versão
■ Urgente	Mover SECRET_KEY para .env (python-dotenv)	1.0.1
■ Urgente	Restringir CORS para domínio de produção	1.0.1
■ Alta	Implementar rate limiting com slowapi	1.1
■ Alta	Configurar HTTPS com Let's Encrypt	1.1
■ Média	Adicionar logging estruturado (JSON)	1.1
■ Média	Rodar pip-audit e npm audit em CI	1.1
■ Baixa	Adicionar testes de segurança automatizados (bandit)	1.2
■ Baixa	Implementar CSP headers no frontend	1.2